Web aplikacije, Predavanje 02

Arhitektura backend aplikacije

Fakultet informatike, Sveučilište Jurja Dobrile u Puli

izv. prof. dr. sc. Nikola Tanković

U prvom predavanju pratili smo jedan zahtjev, `GET /products`, od preglednika do poslužitelja i natrag. Handler je bio kratak jer je samo čitao podatke. Danas dodajemo naručivanje, prvi zahtjev koji mijenja stanje. Jedan endpoint odjednom mora čitati tijelo poruke, provjeriti unos, pronaći cijene, provjeriti i umanjiti zalihu, spremiti narudžbu, obavijestiti kupca i odabrati HTTP status. Pitanje predavanja glasi: **kamo smjestiti svaki od tih poslova da bi kod ostao razumljiv, provjerljiv i promjenjiv?**

Kod uz predavanje nalazi se u mapi `code/02-backend-architecture`. Polazi od kopije `code/01-introduction` i ne koristi vanjske pakete, kako bi se vidjelo što kasnije preuzima framework. Na kraju isti dizajn pokazujemo u Fastifyju.

1 Prošli put: kako biste rasporedili POST /orders?

Vrijeme za nastavu: 1–4 min (3 min).

Prvo predavanje završilo je otvorenim zadatkom: poslove narudžbe (provjera unosa, cijena, zaliha, spremanje) trebalo je rasporediti između preglednika, backenda i baze. Većina obveznih poslova završila je u backendu, jer klijent šalje što hoće. Dodatni uvjet bio je da baza kasni, a korisnik ponovno klikne „Naruči”.

Danas počinjemo od tih prijedloga. Studenti su vjerojatno predložili različite podjele: jednu funkciju, nekoliko funkcija ili „slojeve” poznate iz tutoriala. Svi su legitimni polazni prijedlozi. Na kraju sata trebali bismo moći reći ne samo *kako* smo podijelili kod nego i *zašto* baš tako, te što bismo dobili ili izgubili drukčijom podjelom.

Ponovljeni klik i istodobni zahtjevi ostaju otvoreni problem. Danas ih samo prepoznajemo; ispravno rješenje traži transakcije (5. predavanje) i idempotentnost (9. predavanje).

Prošli put: poslove narudžbe rasporedili smo između preglednika, backenda i baze. Većina je završila u backendu.

Danas: kamo ih smjestiti unutar backenda i zašto?

Za nastavnika: Zamolite dva ili tri para da u jednoj rečenici kažu kako su podijelili posao. Zapišite prijedloge na ploču; vratit ćete im se kod vježbe „Gdje pripada pravilo?”.

Očekivano obrazloženje: Nema jednog točnog odgovora. Prihvatite i „sve u jednoj funkciji” kao polazište; danas ćemo vidjeti kada to prestaje biti dovoljno.

2 Ishodi i predznanje

Vrijeme za nastavu: 4–6 min (2 min).

Ovo predavanje ne uči određeni framework. Uči kako podijeliti backend kod tako da svaki dio odgovara na jedno pitanje: kako izgleda HTTP poruka, koja poslovna pravila vrijede, gdje su podaci i koje vanjske usluge sudjeluju. Ista podjela postoji u Fastifyju, Expressu, Spring Bootu, ASP.NET Coreu ili Djangu, samo pod drugim imenima.

Nakon predavanja moći ćete (u zagradi je razina revidirane Bloomove taksonomije):

- **objasniti** zašto zajedničko stanje ne smije biti u memoriji procesa *(razumijevanje)*
- **konfigurirati** backend iz okruženja i uredno ga ugasiti *(primjena)*
- **testirati** pravila servisa s lažnim ovisnostima, bez HTTP-a *(primjena)*
- **razložiti** `POST /orders` na usmjeravanje, HTTP sloj, pravila, podatke i vanjske usluge *(analiza)*
- **obrazložiti** u koji dio koda pripada novo pravilo *(vrednovanje)*

Predznanje: predavanje 01 (zahtjev, odgovor, status, handler, proces, port) i JavaScript (`async`/`await`, moduli `import`/`export`).

Za nastavnika: Ishode samo pročitajte. Naglasite da je cilj obrazložena odluka, a ne točan broj slojeva ili mapa.

3 Novi zahtjev: POST /orders

Vrijeme za nastavu: 6–9 min (3 min).

Dosad je backend samo vraćao podatke. Narudžba je drugačija: klijent šalje podatke u **tijelu zahtjeva**, a backend mijenja stanje (zaliha se smanjuje, nastaje nova narudžba). Metoda `POST` u HTTP-u znači da poslani sadržaj obrađuje ciljani resurs prema vlastitoj semantici; za nas to znači „stvori narudžbu”. Detaljna pravila metoda i statusa razrađujemo u 3. predavanju.

Klijent u tijelu šalje e-poštu kupca i stavke, svaku s identifikatorom proizvoda i količinom. Zaglavlje `Content-Type: application/json` govori da je tijelo JSON. Cijenu klijent **ne šalje**: u prvom predavanju vidjeli smo da backend ne smije povjerovati iznosu koji dolazi od klijenta.

```
POST /orders HTTP/1.1
Host: 127.0.0.1:3000
Content-Type: application/json

{"email": "ana@primjer.hr", "items": [{"productId": 7, "quantity": 2}]}
```

Uspješan odgovor ima status `201 Created` i opis nastale narudžbe:

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"id": 1, "email": "ana@primjer.hr", "items": [{"productId": 7, "name": "Bilježnica", "qua
```

Odgovor je izvadak stvarnog odgovora nastavnog backenda; izostavljeno je polje `createdAt`.

Izvori: RFC 9110, odjeljak 9.3.3 POST; RFC 9110, 15.3.2 201 Created.

> **Za nastavnika:** Pitajte: što je novo u odnosu na `GET /products`? Očekivano: tijelo zahtjeva, promjena stanja, status 201. Ako pitaju za `Location` zaglavlje, recite da ga uvodimo s `GET /orders/{id}` u 3. predavanju.

4 Što sve treba napraviti?

> **Vrijeme za nastavu:** 9–11 min (2 min).

Prije pisanja koda nabrojimo poslove. Za jedan `POST /orders` backend mora:

1. pročitati tijelo zahtjeva i pretvoriti JSON u objekt
2. provjeriti je li unos ispravan (e-pošta, stavke, količine)
3. pronaći svaki proizvod i njegovu cijenu
4. provjeriti ima li dovoljno na zalihi
5. umanjiti zalihu i spremiti narudžbu
6. obavijestiti kupca e-poštom
7. odabrati status i poslati odgovor

Poslovi nisu iste vrste. Prvi i zadnji ovise o HTTP-u. Drugi i četvrti su pravila trgovine. Treći i peti ovise o tome gdje su podaci. Šesti ovisi o vanjskoj usluzi koju ne kontroliramo. Ta razlika bit će temelj podjele.

> **Za nastavnika:** Pitajte studente koji bi od ovih poslova ostao isti da umjesto web stranice narudžbe stižu iz mobilne aplikacije ili iz skripte za uvoz. Očekivano: 2–6 ostaju isti; mijenja se samo ono što ovisi o HTTP-u.

5 Naivan handler

Vrijeme za nastavu: 11–15 min (4 min).

Prva verzija, kakvu bi napisao početnik ili generirao AI asistent na upit „napiši POST /orders", obavlja sve poslove u jednoj funkciji. Nalazi se u `code/02-backend-architecture/naive/server-nai` i pokreće se s `npm run naive`. Pomoćna funkcija `send` postavlja status i zaglavlje te šalje JSON.

```
// NAMJERNO NAIVNO: sve u jednom handleru
if (req.method === "POST" && req.url === "/orders") {
  let text = "";
  for await (const chunk of req) text += chunk;
  const body = JSON.parse(text);
  if (!body.email || !Array.isArray(body.items) || body.items.length === 0)
    return send(res, 400, { error: "Neispravna narudžba" });
  let total = 0;
  for (const item of body.items) {
    const product = products.find((p) => p.id === item.productId);
    if (!product || product.stock < item.quantity)
      return send(res, 409, { error: "Nema na zalihi" });
    product.stock -= item.quantity;
    total += product.price * item.quantity;
  }
  const order = { id: orders.length + 1, email: body.email, items: body.items, total };
  orders.push(order);
  console.log(`Šaljem e-poštu na ${body.email}: narudžba ${order.id}`);
  return send(res, 201, order);
}
```

Kod je kratak, čitljiv i za ispravnu narudžbu radi. Cijenu ispravno uzima iz popisa proizvoda, a ne iz zahtjeva. Upravo zato je dobar primjer: problemi nisu očiti na prvi pogled. `for await` čita tijelo zahtjeva dio po dio dok ne stigne cijelo.

> **Za nastavnika:** Dajte minutu da pročitaju kod. Ne otkrivajte probleme; sljedeći slajd traži predviđanje. Ako pitaju zašto je `products` niz: baza dolazi u 4. predavanju.

6 Predvidite: četiri zahtjeva

Vrijeme za nastavu: 15–18 min (3 min).

Naivni poslužitelj tek je pokrenut; bilježnica (proizvod 7) ima zalihu 5. Šaljemo redom četiri zahtjeva, svaki s ispravnom e-poštom. Na temelju koda s prethodnog slajda predvidite status i posljedicu.

Stavke u tijelu	Status?	Što se dogodi?
`[{"productId": 7, "quantity": 2}]`	?	?
`[{"productId": 7, "quantity": "2"}]`	?	?
`[{"productId": 7, "quantity": -3}]`	?	?
tijelo `{"email":` (prekinut JSON)	?	?

Za nastavnika: Parovi neka pišu predviđanja dvije minute. Odgovori su na sljedećem slajdu; ne otkrivajte ih prije.

Očekivano obrazloženje: (1) 201, ukupno 7, zaliha 5 → 3. (2) 201: `3 < "2"` je u JavaScriptu `false`, a `stock -= "2"` pretvara tekst u broj; zaliha 3 → 1, a narudžba sadrži tekst `"2"`. (3) 201 s ukupnim iznosom −10,50 €; zaliha raste 1 → 4. (4) `JSON.parse` baca iznimku koju nitko ne hvata i **cijeli proces se ruši**; poslužitelj ne odgovara nikome dok ga netko ponovno ne pokrene.

7 Provjerimo

Vrijeme za nastavu: 18–21 min (3 min).

Zahtjeve smo poslali naivnom poslužitelju (Node.js 20.11, lokalno). Rezultati:

Stavke	Status	Posljedica
`quantity: 2`	`201`	ukupno 7 €, zaliha 5 → 3
`quantity: "2"`	`201`	tekst prolazi kao broj, zaliha 3 → 1
`quantity: -3`	`201`	ukupno **−10,50 €**, zaliha **raste** 1 → 4
prekinut JSON	—	**proces se ruši**, svi ostali korisnici ostaju bez usluge

Postoji i peti, manje vidljiv problem. Ako narudžba ima dvije stavke, a za drugu nema zalihe, prva je zaliha već umanjena prije odgovora `409`. Provjerili smo: nakon odbijene narudžbe „2 olovke + 5 ruksaka” na zalihi je nestalo dvije olovke, iako narudžbe nema.

Nijedan od ovih problema ne traži „bolju arhitekturu” da bi se popravio; svaki se može zakrpati još jednim `if`. Ali svaka zakrpa ulazi u istu funkciju, koja tako postaje sve duža i teža za provjeru. Arhitektura nam ne popravlja greške sama; pomaže da svako pravilo ima jedno očito mjesto i da ga možemo provjeriti bez pokretanja cijelog poslužitelja.

Za nastavnika: Demonstracija: `npm run naive` u `code/02-backend-architecture`, zatim četiri `curl` naredbe iz README-a ili iz bilješki. Nakon četvrte pokažite terminal: `SyntaxError` i proces je završio. Pitajte: koliko je korisnika pogođeno jednim neispravnim zahtjevom? Očekivano: svi.

8 Jedan handler, sedam poslova

Vrijeme za nastavu: 21–24 min (3 min).

Vratimo se popisu poslova i označimo ih u naivnom kodu. Dijagram pokazuje svaki redak, posao koji obavlja i razlog zbog kojeg bi se taj redak mogao promijeniti.



Figure 1: Naivni handler: sedam redaka koda, sedam poslova i sedam različitih razloga za promjenu.

Razlozi za promjenu dolaze iz različitih smjerova. Prelazak na bazu (4. predavanje) mijenja retke s `products.find` i `orders.push`. Novo pravilo trgovine, primjerice „najviše 5 komada po proizvodu”, mijenja provjere. Pravi pružatelj e-pošte mijenja redak s `console.log`. Dogovor o statusima (3. predavanje) mijenja `send`. Kad su svi ti razlozi u istoj funkciji, svaka promjena traži čitanje i ponovno testiranje cijele funkcije, a dvije osobe koje rade na različitim promjenama mijenjaju isti kod.

Za nastavnika: Pitajte: koji bi redak promijenio programer koji uvodi PostgreSQL, a koji onaj koji uvodi popust? Očekivano: različiti retci, ista funkcija. To je problem koji rješavamo.

9 Razdvajanje odgovornosti

Vrijeme za nastavu: 24–28 min (4 min).

Ideju da se kod koji odgovara na različita pitanja drži odvojeno opisao je Edsger W. Dijkstra 1974. pod imenom *separation of concerns*. Za backend to znači da odvajamo četiri vrste pitanja:

Dio	Na koje pitanje odgovara	U našem kodu
Usmjeravanje	koji kod obrađuje ovu metodu i putanju?	`src/app.mjs`
HTTP sloj (handler)	kako čitam zahtjev i koji status vraćam?	`order-handlers.mjs`
Servis	koja pravila vrijede za narudžbu?	`order-service.mjs`
Pristup podacima i vanjske usluge	gdje su podaci i kako šaljem e-poštu?	`memory-catalog.mjs`, `notifier.mjs`

Definicija 2.1

Razdvajanje odgovornosti (*separation of concerns*): kod koji se mijenja iz različitih razloga držimo u zasebnim dijelovima, a oni surađuju preko malog broja jasno imenovanih funkcija.

Podjela nije cilj sama po sebi. Svaki novi dio dodaje datoteku, ime i poziv. Isplati se kad dijelovi stvarno imaju različite razloge za promjenu ili kad ih želimo zasebno provjeriti.

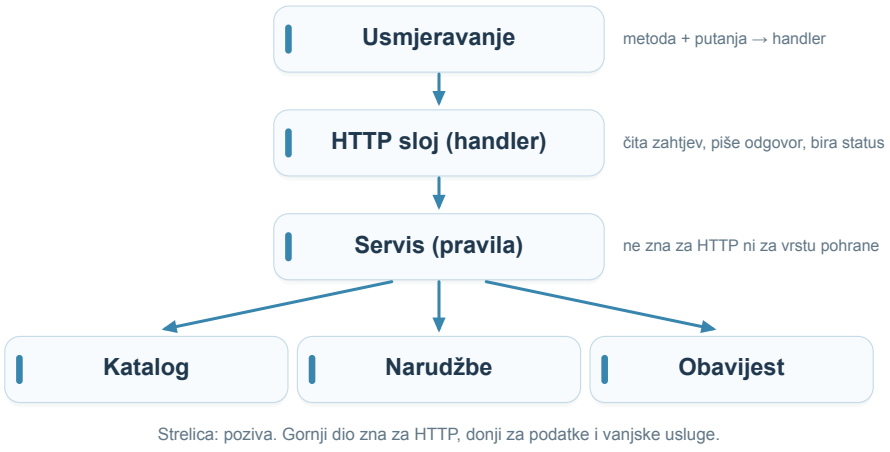


Figure 2: Odgovornosti nakon podjele: strelica znači „poziva". Gornji dijelovi znaju za HTTP, donji za podatke i vanjske usluge.

Izvori: Dijkstra, *On the role of scientific thought*, EWD447 (1974); Fowler, *PresentationDomainDataLayering* (2015).

> **Za nastavnika:** Usporedite s prijedlozima s početka sata. Naglasite da imena („controller", „service", „repository") variraju po frameworku; važna su pitanja na koja dio odgovara.

10 Usmjeravanje: metoda i putanja određuju handler

> **Vrijeme za nastavu:** 28–30 min (2 min).

U prvom predavanju **ruta** je povezivala metodu i putanju s funkcijom. Kad ruta ima više, korisno ih je zapisati na jednom mjestu kao tablicu. Usmjeravanje (*routing*) ne sadrži pravila trgovine; samo bira handler i odgovara na zahtjeve koje nijedan handler ne prihvaća.

```
// src/app.mjs (skraćeno)
const routes = {
  "GET /": serveIndex,
  "GET /products": productHandlers.list,
  "POST /orders": orderHandlers.create,
};
const handler = routes[`${req.method} ${url.pathname}`];
if (handler) return await handler(req, res, url);
// putanja postoji s drugom metodom → 405, inače → 404
```

Isto mjesto hvata neočekivane greške: ako handler baci grešku koju nitko ne obradi, `app.mjs` je zapisuje u terminal i vraća `500` umjesto da se proces sruši. Time je riješen četvrti slučaj s naivnog poslužitelja. Neispravan JSON sada daje `400 Tijelo nije valjan JSON`, a ostali korisnici ništa ne primjećuju.

Frameworci poput Fastifyja i Expressa nude isto, uz parametre u putanji (`/orders/:id`) i brže pretraživanje ruta.

> **Za nastavnika:** Pokažite `GET /orders` → `405` i `GET /nesto` → `404` na pokrenutom backendu (`npm start`). Pitajte zašto je razlika korisna klijentu. Očekivano: 405 kaže da putanja postoji, ali ne s tom metodom.

11 Handler: HTTP ulaz i izlaz

> **Vrijeme za nastavu:** 30–33 min (3 min).

Handler je granica između HTTP-a i ostatka aplikacije. Prevodi u oba smjera: HTTP zahtjev u poziv funkcije, a rezultat ili grešku u HTTP odgovor. U nekim frameworcima zove se *controller*.

```
// src/orders/order-handlers.mjs
async create(req, res) {
  const body = await readJson(req);              // 415 ili 400 ako tijelo nije JSON
  try {
    const order = await orderService.placeOrder(body);
    return sendJson(res, 201, order);
  } catch (error) {
    if (error instanceof ValidationError) return sendJson(res, 400, { error: error.message
    if (error instanceof OutOfStockError) return sendJson(res, 409, { error: error.message
    throw error;                                 // neočekivano → app.mjs vraća 500
  }
}
```

Handler ne zna kako se računa ukupni iznos ni gdje su proizvodi. Zna samo HTTP: kako pročitati tijelo, koju funkciju pozvati i koji status pripada kojem ishodu. Funkcija `readJson` provjerava `Content-Type` i parsira tijelo; neispravno tijelo pretvara u grešku sa statusom.

> **Za nastavnika:** Pitajte: što se mijenja u ovom kodu ako odlučimo da nedovoljna zaliha vraća 422 umjesto 409? Očekivano: samo jedan redak u handleru; servis ostaje isti.

12 Servis: poslovna pravila bez HTTP-a

> **Vrijeme za nastavu:** 33–37 min (4 min).

Servis sadrži pravila naručivanja: što je ispravan unos, odakle cijena, kada nema dovoljno zalihe, što se sprema i kada se šalje obavijest. Prima običan JavaScript objekt i vraća običan objekt ili baca grešku. Ne poznaje `req`, `res` ni statuse.

```
// src/orders/order-service.mjs (skraćeno)
async placeOrder(input) {
  const { email, items } = validate(input);              // baca ValidationError
  const lines = [];
  for (const { productId, quantity } of items) {
    const product = await catalog.findById(productId);
    if (!product) throw new ValidationError(`Proizvod ${productId} ne postoji`);
    if (product.stock < quantity) throw new OutOfStockError(`Nema dovoljno: ${product.name
    lines.push({ productId, name: product.name, quantity, unitPrice: product.price });
  }
  for (const line of lines) await catalog.decreaseStock(line.productId, line.quantity);
  const order = await orders.save({ email, items: lines, total: sum(lines) });
  await notifyQuietly(order);                            // greška obavijesti ne ruši nar
  return order;
}
```

Usporedimo s naivnom verzijom. Funkcija `validate` traži cjelobrojnu količinu barem 1, pa `"2"` i `-3` daju `ValidationError`. Zaliha se umanjuje tek kad su **sve** stavke provjerene, pa odbijena narudžba više ne troši zalihu. Isti proizvod ne smije se pojaviti dvaput u istoj narudžbi, jer bi inače svaka stavka zasebno prošla provjeru zalihe.

Jedan problem ostaje i označen je komentarom u kodu: između provjere zalihe i njezina umanjenja postoji `await`, pa dva istodobna zahtjeva mogu oba proći provjeru za posljednji ruksak. To je *check-then-act* utrka koju rješavamo u 5. predavanju. Podjela koda ne rješava konkurentnost, ali ju je lakše pronaći kad je pravilo na jednom mjestu.

> **Za nastavnika:** Pitajte: može li se ovaj servis pozvati iz skripte za uvoz narudžbi iz CSV-a, bez HTTP-a? Očekivano: da, jer ne ovisi o `req` i `res`. Komentar o utrci samo spomenite i najavite 5. predavanje.

13 Repozitorij: pristup podacima

> **Vrijeme za nastavu:** 37–40 min (3 min).

Servis traži proizvode preko objekta `catalog` s tri funkcije: `list`, `findById` i `decreaseStock`. Danas su podaci niz u memoriji procesa. U 4. predavanju isti će objekt čitati iz PostgreSQL-a. Dio koda koji skriva gdje i kako su podaci spremljeni zove se **repozitorij** (*repository*) ili sloj pristupa podacima.

```
// src/products/memory-catalog.mjs (skraćeno)
export function createMemoryCatalog() {
  const products = [{ id: 7, name: "Bilježnica", price: 3.5, stock: 5 }, /* … */];
  return {
    async list() { return products.map(({ id, name, price }) => ({ id, name, price })); },
    async findById(id) { const p = products.find((x) => x.id === id); return p ? { ...p }
    async decreaseStock(id, quantity) { products.find((x) => x.id === id).stock -= quantit
  };
}
```

Funkcije su `async` iako niz ne traži čekanje. Razlog je sučelje: baza će odgovarati asinkrono, a servis ne smije ovisiti o tome je li implementacija trenutačno brza. `findById` vraća kopiju (`{ ...p }`), pa servis ne može slučajno promijeniti zalihu mimo `decreaseStock`. Lista ne izlaže zalihu klijentima; to je odluka o javnim poljima iz prvog predavanja.

Izvori: Fowler, *Patterns of Enterprise Application Architecture*: Repository.

> **Za nastavnika:** Pitajte: što bi se dogodilo da `findById` vraća izvorni objekt umjesto kopije? Očekivano: kod izvan kataloga mogao bi mijenjati zalihu bez `decreaseStock`; teško je pronaći tko je promijenio podatak.

14 Obavijest: vanjska usluga iza funkcije

> **Vrijeme za nastavu:** 40–42 min (2 min).

E-pošta je **vanjska usluga**: drugi sustav, koji može biti spor ili nedostupan. Servis ga poziva preko objekta `notifier` s jednom funkcijom, `orderPlaced(order)`. Nastavni backend e-poštu ne šalje; ispisuje redak u terminal. Varijabla okruženja `NOTIFIER` odabire način rada: `console`, `slow` (3 s čekanja) ili `fail` (usluga ne radi).

Važna je odluka što učiniti kad obavijest ne uspije. Narudžba je u tom trenutku već spremljena. Kad bismo klijentu vratili `500`, poruka bi lagala: korisnik bi mislio da narudžba nije uspjela i možda ponovio kupnju. Zato servis grešku obavijesti hvata i zapisuje, a klijent dobiva `201`. Provjerili smo s `NOTIFIER=fail`: odgovor je `201`, a u terminalu piše „Obavijest za narudžbu 1 nije poslana".

Odluka ima cijenu. Kupac možda neće dobiti e-poštu, a s `NOTIFIER=slow` svaki odgovor kasni tri sekunde jer servis čeka obavijest. Oba problema vode prema 8. predavanju, gdje e-poštu izdvajamo u pozadinski posao.

> **Za nastavnika:** Pokažite `NOTIFIER=fail npm start` i jednu narudžbu. Pitajte: je li ispravno vratiti 201? Prihvatite raspravu; ključ je da odgovor ne smije tvrditi nešto netočno o spremljenoj narudžbi.

15 POST /orders kroz dijelove

> **Vrijeme za nastavu:** 42–45 min (3 min).

Pratimo isti zahtjev kao u prvom predavanju, ali sada unutar procesa. Strelice su pozivi funkcija, a ne mrežne poruke: svi dijelovi rade u istom procesu.

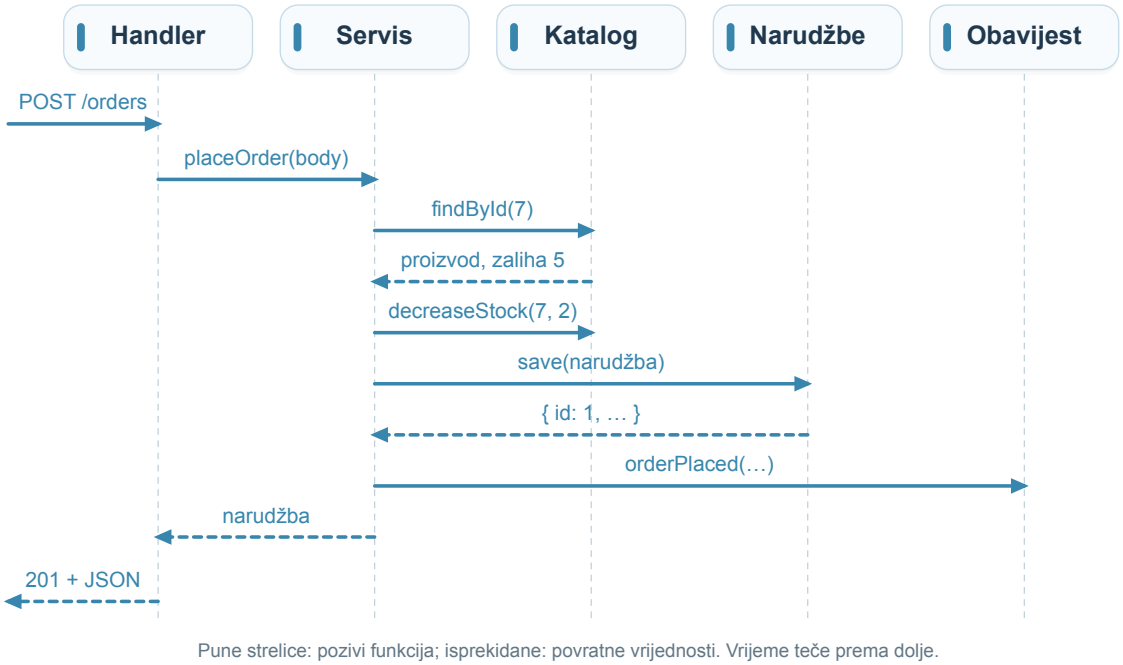


Figure 3: Slijed `POST /orders` kroz handler, servis, katalog, pohranu narudžbi i obavijest. Pune strelice su pozivi, isprekidane povratne vrijednosti; vrijeme teče prema dolje.

1. Usmjeravanje predaje zahtjev handleru, koji čita tijelo i poziva `placeOrder`.
2. Servis za svaku stavku pita katalog za proizvod i provjerava zalihu.
3. Tek nakon provjere svih stavki umanjuje zalihu i sprema narudžbu.
4. Poziva obavijest, a narudžbu vraća handleru.
5. Handler šalje `201` i JSON.

Za greške put je kraći: `ValidationError` ili `OutOfStockError` prekidaju servis prije bilo kakve promjene, a handler ih pretvara u status.

> **Za nastavnika:** Pitajte: u kojem bi koraku `NOTIFIER=slow` produljio odgovor? Očekivano: kod `orderPlaced`, jer servis čeka obavijest prije povratka.

16 Vježba: gdje pripada pravilo?

> **Vrijeme za nastavu:** 45–49 min (4 min).

Za svako pravilo odredite dio koda: usmjeravanje, HTTP sloj, servis, katalog/pohrana ili konfiguracija. Ponegdje je obranjivo više odgovora; važno je obrazloženje.

Pravilo	Gdje?
Tijelo mora imati `Content-Type: application/json`.	?

Pravilo	Gdje?
Količina je cijeli broj veći od nule.	?
Cijena stavke uzima se iz kataloga.	?
Nedovoljna zaliha vraća status `409`.	?
Proizvod se traži po identifikatoru.	?
Novo: najviše 5 komada istog proizvoda po narudžbi.	?
Poslužitelj sluša na portu 3000.	?

Za nastavnika: Parovi dvije minute, zatim prolaz kroz tablicu.

Očekivano obrazloženje: Content-Type → HTTP sloj (`readJson`). Količina → servis; obranjivo i dodatno u HTTP sloju kao provjera oblika (Fastify sheme, 3. predavanje), ali pravilo mora vrijediti i bez HTTP-a. Cijena → servis. 409 → HTTP sloj; samo pravilo „nema dovoljno" je u servisu. Traženje po id-u → katalog. Najviše 5 komada → servis. Port → konfiguracija. Usporedite s prijedlozima s početka sata.

17 Greške prelaze granicu

Vrijeme za nastavu: 49–52 min (3 min).

Servis ne vraća HTTP statuse; baca greške koje opisuju što se dogodilo u pravilima. Handler ih prevodi u HTTP. Tako isti servis, pozvan iz testa ili skripte, baca istu grešku, a samo HTTP sloj odlučuje kako je prikazati klijentu.

Izvor greške	Greška	HTTP odgovor
`readJson`	tijelo nije JSON	`415` ili `400`
servis	`ValidationError`	`400`
servis	`OutOfStockError`	`409`
bilo što neočekivano	ostale greške	`500`, zapis u terminal (`app.mjs`)

Neočekivana greška ne smije otkriti unutarnje detalje klijentu. Poruka je općenita („Neočekivana greška poslužitelja"), a detalj ide u zapis koji vidi samo onaj tko održava sustav. Oblik grešaka u odgovoru (polja, kodovi) dogovaramo u 3. predavanju; sigurnosni aspekt otkrivanja detalja u 7.

Svi statusi u tablici provjereni su `curl` zahtjevima prema pokrenutom backendu.

Za nastavnika: Pitajte: zašto servis ne baca odmah objekt `{ status: 409 }`? Očekivano: tada bi servis ovisio o HTTP-u; skripta za uvoz ne bi imala što raditi sa statusom.

18 Smjer ovisnosti

Vrijeme za nastavu: 52–55 min (3 min).

Kod iz jednog modula koji uvozi ili poziva drugi modul **ovisi** o njemu: promjena u drugom modulu može ga pokvariti. Bitno je u kojem smjeru idu te ovisnosti. HTTP sloj ovisi o servisu, jer ga poziva. Servis bi trebao ovisiti samo o funkcijama koje mu trebaju (`findById`, `save`, `orderPlaced`), a ne o konkretnoj datoteci s nizom, o `node:http` ili o određenom pružatelju e-pošte.

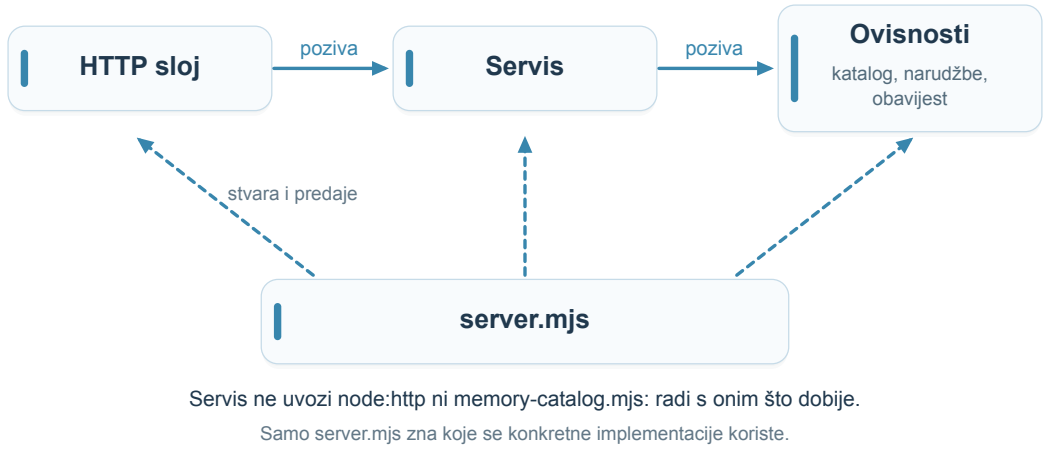


Figure 4: Smjer ovisnosti: HTTP sloj poziva servis, servis poziva ovisnosti koje je dobio. Samo `server.mjs` stvara konkretne implementacije i predaje ih.

Definicija 2.2

Smjer ovisnosti: poslovna pravila ne ovise o detaljima poput HTTP-a, baze ili e-pošte. Detalji ovise o pravilima ili im se predaju izvana.

Posljedica: kad promijenimo detalj (bazu, framework, pružatelja e-pošte), servis se ne mijenja. Kad promijenimo pravilo, ne diramo HTTP ni bazu. Ovo je jezgra ideja koje se u literaturi zovu *dependency inversion*, heksagonalna arhitektura ili *ports and adapters*; danas ne trebamo ta imena, nego smjer strelica.

Izvori: Fowler, *Inversion of Control Containers and the Dependency Injection pattern* (2004).

Za nastavnika: Pitajte: gdje je u `order-service.mjs` riječ `import`? Očekivano: nigdje. `order-service.mjs` ne uvozi nijedan modul; greške definira sam, a ovisnosti dobiva kao parametre. Pokažite to u datoteci.

19 Ubrizgavanje ovisnosti

Vrijeme za nastavu: 55–58 min (3 min).

Kako servis dobiva katalog ako ga ne uvozi? Dobiva ga kao parametar pri stvaranju. To se zove **ubrizgavanje ovisnosti** (*dependency injection*): umjesto da dio koda sam bira i stvara ono o čemu ovisi, netko izvana mu to preda. Za to ne treba nikakva biblioteka; dovoljna je funkcija s parametrima.

```
// Bez ubrizgavanja: servis sam bira implementacije
import { products } from "./products-array.mjs";
import { sendEmail } from "./smtp.mjs";

// S ubrizgavanjem: servis dobiva ovisnosti (order-service.mjs)
export function createOrderService({ catalog, orders, notifier }) { /* … */ }

// server.mjs: jedino mjesto koje bira konkretne implementacije
const catalog = createMemoryCatalog();
const orderService = createOrderService({
  catalog,
  orders: createMemoryOrders(),
  notifier: createNotifier({ mode: config.notifier }),
});
```

Mjesto koje sastavlja aplikaciju zove se *composition root*. Kod nas je to `server.mjs`. Isti katalog dobivaju i servis narudžbi i handler proizvoda, pa obje rute vide istu zalihu. Spring i ASP.NET Core imaju spremnike (*containers*) koji ovo rade automatski; ideja je ista.

Izvori: Fowler, *Inversion of Control Containers and the Dependency Injection pattern* (2004).

Za nastavnika: Pitajte: kad bi servis i handler proizvoda svaki stvorili svoj katalog, što bi `GET /products` pokazivao nakon narudžbe? Očekivano: dvije zasebne zalihe; isti problem kao dva procesa, samo unutar jednog.

20 Dokaz: test pravila bez HTTP-a

Vrijeme za nastavu: 58–61 min (3 min).

Prva konkretna korist: pravila možemo provjeriti bez pokretanja poslužitelja, bez mreže i bez e-pošte. Test sam složi servis s ovisnostima koje mu odgovaraju, uključujući lažnu obavijest (*fake*) koja samo bilježi poziv.

```
// test/order-service.test.mjs (skraćeno)
test("cijena dolazi iz kataloga, ne iz zahtjeva", async () => {
  const service = createOrderService({
    catalog: createMemoryCatalog(),
    orders: createMemoryOrders(),
    notifier: { orderPlaced: async () => {} },     // lažna e-pošta
  });
  const order = await service.placeOrder({
    email: "a@b.hr", items: [{ productId: 7, quantity: 2, price: 0.01 }],
  });
  assert.equal(order.total, 7);
});
```

Naredba `npm test` pokreće pet takvih testova ugrađenim modulom `node:test` za oko 60 ms: cijena iz kataloga, nedovoljna zaliha ne mijenja zalihu, neispravna količina, neuspjela obavijest ne poništava narudžbu i jedna obavijest po narudžbi. Rezultat lokalno: `# pass 5`, `# fail 0`.

Test koji prolazi ne dokazuje da je kod ispravan u svim uvjetima. Ovi testovi, na primjer, ne otkrivaju utrku dvaju istodobnih zahtjeva. Testiranje detaljnije obrađujemo u 11. predavanju.

Izvori: Node.js: Test runner (`node:test`).

> **Za nastavnika:** Pokrenite `npm test` uživo. Zatim namjerno pokvarite pravilo (npr. `unitPrice: item.price ?? product.price`) i pokažite koji test pada.

21 Zamjena implementacije

> **Vrijeme za nastavu:** 61–63 min (2 min).

Servis zna samo za funkcije `findById` i `decreaseStock`. Sve što ih ostvaruje može se predati umjesto memorijskog kataloga: danas niz, u testu lažni katalog, u 4. predavanju PostgreSQL. Servis se pri tome ne mijenja.

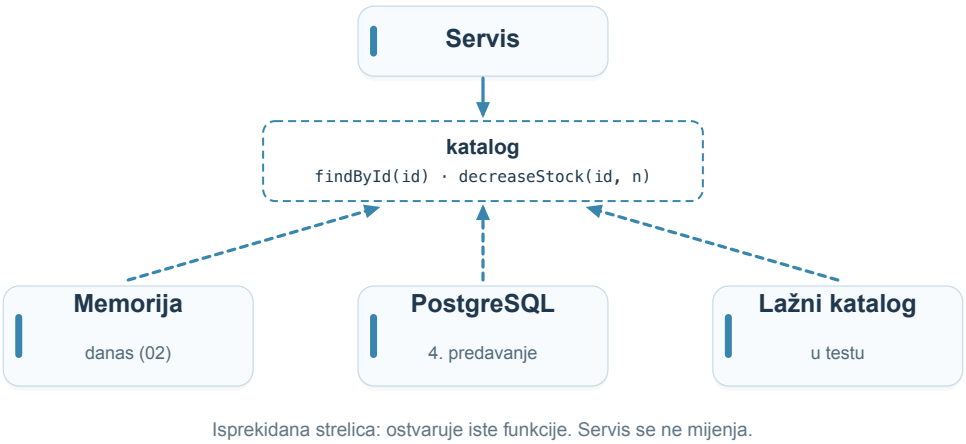


Figure 5: Zamjena implementacije: memorijski katalog, PostgreSQL i lažni katalog u testu ostvaruju iste funkcije; servis ostaje isti.

JavaScript ne provjerava ima li predani objekt sve potrebne funkcije; to otkrivamo tek pri pozivu ili testom. Jezici poput TypeScripta, Jave ili C# to provjeravaju prije pokretanja pomoću sučelja (*interface*). Ideja je ista, samo je provjera ranija.

> **Za nastavnika:** Najavite: u 4. predavanju pišemo `createPostgresCatalog()` s istim funkcijama i mijenjamo jedan redak u `server.mjs`.

22 Ne treba sloj za svaku funkciju

> **Vrijeme za nastavu:** 63–66 min (3 min).

Podjela ima cijenu i lako se pretjera. Česta je greška, pogotovo u generiranom kodu, da svaki endpoint dobije kontroler, servis i repozitorij iako ništa ne radi osim prosljeđivanja:

```
// Slojevi koji ništa ne dodaju
class ProductService    { list() { return this.repository.list(); } }
class ProductController { list() { return this.productService.list(); } }
```

`GET /products` nema poslovnih pravila: samo vraća javna polja kataloga. Zato u našem kodu handler proizvoda izravno poziva katalog, bez servisa. Servis narudžbi postoji jer ima pravila koja želimo testirati i zaštititi od HTTP detalja.

Pitanje za svaki novi dio glasi: koji razlog za promjenu ili koju provjeru on izdvaja? Ako ne postoji odgovor, novi dio je samo dodatna datoteka koju treba čitati. Kad se pravilo pojavi (npr. skriveni proizvodi za neprijavljene korisnike), servis proizvoda dodajemo tada.

> **Za nastavnika:** Pitajte: kad biste dodali `ProductService`? Očekivano: kad se pojavi pravilo koje ne pripada ni HTTP-u ni pohrani.

23 Konfiguracija dolazi iz okruženja

Vrijeme za nastavu: 66–69 min (3 min).

Isti kod pokrećemo na različitim mjestima: na prijenosniku, na testnom poslužitelju, u produkciji. Ono što se razlikuje među tim okruženjima, poput porta, adrese, načina slanja e-pošte i pristupnih ključeva, zove se **konfiguracija**. Ne pišemo je u kod, nego je čitamo iz **varijabli okruženja**: imenovanih vrijednosti koje operacijski sustav predaje procesu pri pokretanju.

Varijabla	Lokalno	Produkcija (primjer)
`PORT`	`3000`	dodjeljuje platforma
`HOST`	`127.0.0.1`	`0.0.0.0` (sve mrežne adrese)
`NOTIFIER`	`console`	pravi pružatelj
`EMAIL_API_KEY`	prazno	tajna iz upravitelja tajni

```
// src/config.mjs (skraćeno)
export function loadConfig(env = process.env) {
  const port = Number(env.PORT ?? 3000);
  if (!Number.isInteger(port) || port < 1 || port > 65535)
    throw new Error(`PORT mora biti cijeli broj 1-65535, a dobiven je "${env.PORT}"`);
  return Object.freeze({ port, host: env.HOST ?? "127.0.0.1", /* … */ });
}
```

Konfiguraciju čita jedno mjesto, jednom, pri pokretanju. Ostatak koda dobiva gotov objekt (`Object.freeze` sprječava slučajnu promjenu) i ne čita `process.env` sam. Varijable se postavljaju u naredbi (`PORT=3001 npm start`) ili u datoteci `.env` koju Node od verzije 20.6 može učitati opcijom `--env-file` (`npm run start:env`).

Izvori: The Twelve-Factor App: III. Config; Node.js: `--env-file`.

> **Za nastavnika:** Pokažite `PORT=3001 npm start`. Pitajte: zašto ne čitati `process.env.PORT` izravno u handleru? Očekivano: rasuta konfiguracija se ne može provjeriti na jednom mjestu, a test bi morao mijenjati okruženje procesa.

24 Tajne

Vrijeme za nastavu: 69–71 min (2 min).

Dio konfiguracije su **tajne**: lozinke baze, API ključevi, ključevi za potpisivanje. Tko ih ima, može djelovati u ime aplikacije. Za njih vrijede dodatna pravila:

- **ne u kodu**: `const EMAIL_API_KEY = "re_8f2a…"` vidi svatko tko vidi kod
- **ne u gitu**: `.env` je u `.gitignore`; u repozitoriju je samo `.env.example` s imenima, bez vrijednosti
- **ne u zapisima**: pri pokretanju ispisujemo konfiguraciju, ali za ključ samo „(postavljen)" ili „(nije postavljen)"
- **ako procuri, zamijeni ga**: brisanje iz zadnje verzije ne briše ga iz povijesti gita ni iz tuđih kopija

Provjerili smo: s postavljenim `EMAIL_API_KEY` zapis pri pokretanju prikazuje samo `emailApiKey: '(postavljen)'`.

Izvori: OWASP: Secrets Management Cheat Sheet.

> **Za nastavnika:** Pitajte: student je greškom predao `.env` s ključem na GitHub i odmah ga obrisao novim commitom. Je li problem riješen? Očekivano: ne; ključ je u povijesti i mogao je biti kopiran. Treba ga poništiti kod pružatelja i izdati novi.

25 Neispravna konfiguracija zaustavlja pokretanje

> **Vrijeme za nastavu:** 71–73 min (2 min).

Što ako netko postavi `PORT=abc`? Bez provjere bi se greška pojavila kasnije i na manje očitom mjestu. `loadConfig` zato provjerava vrijednosti i baca grešku, a `server.mjs` ispisuje poruku i završava proces s kodom 1:

```
$ PORT=abc npm start
Neispravna konfiguracija: PORT mora biti cijeli broj 1-65535, a dobiven je "abc"
$ echo $?
1
```

Načelo se zove *fail fast*: bolje je da proces odbije pokretanje odmah, s jasnom porukom, nego da radi napola i prvi kupac u tri ujutro dobije `500`. Izlazni kod različit od nule govori alatu koji pokreće proces (platformi, skripti) da pokretanje nije uspjelo.

Isto vrijedi za zauzeti port. Drugi proces na istom portu sada ispisuje `Pokretanje nije uspjelo: EADDRINUSE (127.0.0.1:3000)` i izlazi s kodom 1, umjesto nečitljivog ispisa neuhvaćene greške.

> **Za nastavnika:** Pokažite `PORT=abc npm start` i `NOTIFIER=email npm start`. Pitajte: što bi se dogodilo s `NOTIFIER=email` da nema provjere? Očekivano: ovisi o kodu; vjerojatno bi se e-pošta tiho ponašala kao zadani način, a greška bi se otkrila tek kad kupci ne dobiju poruke.

26 Životni ciklus procesa

Vrijeme za nastavu: 73–76 min (3 min).

U prvom predavanju proces je bio „jedno pokretanje programa”. Sada pogledajmo njegove faze. `server.mjs` ih izvodi redom: učita i provjeri konfiguraciju, sastavi ovisnosti, i tek tada pozove `listen`. Do tog trenutka proces ne prima zahtjeve, pa nijedan zahtjev ne može stići u napola sastavljenu aplikaciju.



Figure 6: Životni ciklus procesa: pokretanje (konfiguracija, sastavljanje, `listen`), rad i uredno gašenje. Crvene strelice su izlazi s greškom.

Kraj je jednako važan. Proces se gasi kad ga zaustavimo s Ctrl+C (signal `SIGINT`) ili kad ga platforma zaustavlja pri novoj isporuci (obično `SIGTERM`). Ako se tada samo prekine, zahtjevi koji su u tijeku ostaju bez odgovora, a narudžba može biti napola obrađena.

Izvori: Node.js: Signal events; The Twelve-Factor App: IX. Disposability.

> **Za nastavnika:** Povežite s 1. predavanjem: „Program, proces i računalo”. Pitajte: zašto `listen` dolazi zadnji? Očekivano: da prvi zahtjev ne stigne prije nego što su servis i katalog spremni.

27 Uredno gašenje

Vrijeme za nastavu: 76–80 min (4 min).

Uredno gašenje (*graceful shutdown*) znači: prestani primati nove zahtjeve, dovrši one koji su započeli, pa izađi. U Node.js-u to radi `server.close()`. Poslužitelj odmah prestaje

prihvaćati nove veze, a funkcija koju mu predamo poziva se kad su sve postojeće veze završile. Za slučaj da neki zahtjev nikad ne završi, postavljamo gornju granicu od 10 sekundi.

```
// server.mjs (skraćeno)
function shutdown(signal) {
  console.log(`${signal}: ne primam nove zahtjeve, dovršavam započete…`);
  server.close(() => process.exit(0));
  setTimeout(() => process.exit(1), config.shutdownTimeoutMs).unref();
}
process.once("SIGINT", shutdown);
process.once("SIGTERM", shutdown);
```

Ponašanje smo izmjerili lokalno (Node.js 20.11). Uz `NOTIFIER=slow` narudžba traje 3 s. Sekundu nakon slanja narudžbe pritisnuli smo Ctrl+C, a odmah zatim poslali novi zahtjev:

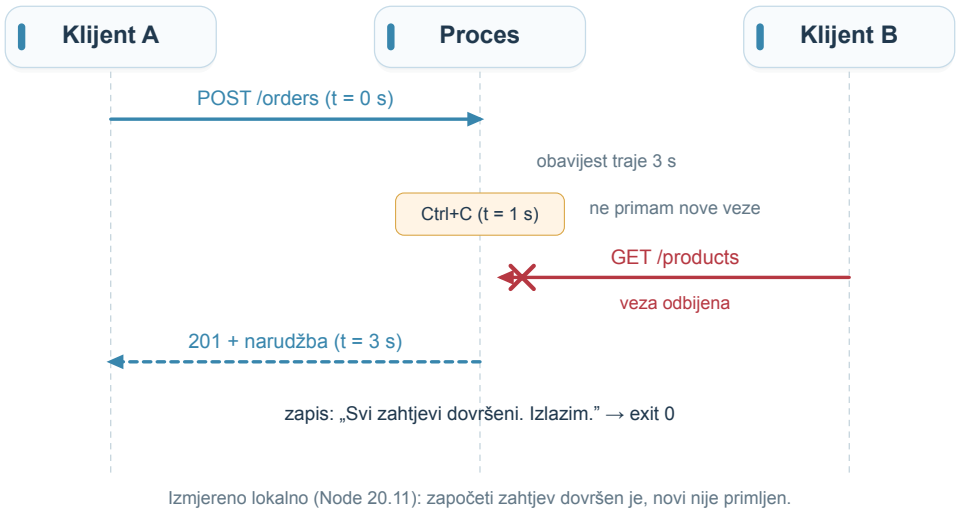


Figure 7: Uredno gašenje, izmjereno: započeta narudžba dovršena je s 201 nakon 3 s, nova veza nakon Ctrl+C odbijena je, a proces je izašao s kodom 0.

`unref()` znači da sam mjerač vremena ne drži proces živim: ako sve završi ranije, proces izlazi odmah. Od Node.js-a 19 `server.close()` zatvara i neaktivne *keep-alive* veze, pa preglednik s otvorenom vezom ne odgađa gašenje.

Uredno gašenje ne pomaže ako proces nestane naglo, primjerice zbog nestanka struje ili signala `SIGKILL`. Tada se gubi sve u memoriji, uključujući narudžbe nastavnog backenda. To je razlog za trajnu pohranu u 4. predavanju.

Izvori: Node.js: `server.close()`; Node.js: `net.Server.close()`.

> **Za nastavnika:** Demonstracija: `NOTIFIER=slow npm start`; u drugom terminalu pošaljite narudžbu (`curl` iz README-a) i odmah pritisnite Ctrl+C u prvom. Pokažite da `curl` dobiva 201 i tek tada proces izlazi. Pitajte: što bi se

dogodilo bez `server.close()`, samo s `process.exit(0)`? Očekivano: veza bi se prekinula bez odgovora; klijent ne bi znao je li narudžba spremljena (izgubljeni odgovor iz 1. predavanja).

28 Dva procesa, jedan ruksak

Vrijeme za nastavu: 80–83 min (3 min).

U prvom predavanju vidjeli smo da proxy može slati zahtjeve na više procesa istog programa. Što to znači za naš backend? Pokrenimo dva procesa, na portovima 3000 i 3001, i u svakom naručimo posljednji ruksak.



Figure 8: Dva procesa, svaki s vlastitom zalihom u memoriji: oba prihvaćaju narudžbu posljednjeg ruksaka i vraćaju 201.

Provjerili smo: oba procesa vraćaju `201`, oba narudžbi daju `id: 1`, a prodana su dva ruksaka iako postoji jedan. Svaki proces ima svoju memoriju, pa svoju zalihu i svoj brojač narudžbi. Nijedan ne zna za drugoga.

Ovo nije utrka u jednom procesu (5. predavanje), nego posljedica toga što stanje koje mora biti zajedničko živi na mjestu koje nije zajedničko. Rješenje je zajednička pohrana, baza, kojoj pristupaju svi procesi.

Za nastavnika: Demonstracija: `npm start` i `PORT=3001 npm start`, zatim ista narudžba ruksaka na oba porta. Pitajte: koliko ruksaka je prodano? Što bi trebao vidjeti proces B?

29 Aplikacijski poslužitelj bez stanja

Vrijeme za nastavu: 83–85 min (2 min).

Iz prethodnog primjera slijedi pravilo za backend koji se može pokrenuti u više primjeraka:

Definicija 2.3

Aplikacijski poslužitelj bez stanja (*stateless*): proces ne čuva u svojoj memoriji podatke koji moraju preživjeti njegovo gašenje ili biti vidljivi drugim procesima; takvo stanje drži u zajedničkoj pohrani.

Ovo nije isto što i „HTTP je stateless" iz prvog predavanja. Tamo se radilo o protokolu, a ovdje o tome gdje proces drži podatke. Proces smije imati privremeno stanje unutar jednog zahtjeva ili pomoćnu predmemoriju koju smije izgubiti. Ne smije biti jedino mjesto za narudžbe, zalihu ili prijave korisnika.

Stanje	Danas	Kamo ide
proizvodi, zaliha, narudžbe	memorija procesa	baza (4. predavanje)
prijava korisnika	—	sesija u zajedničkoj pohrani ili token (6.)
posao koji traje (e-pošta, račun)	čeka se u zahtjevu	red poslova i worker (8.)

Nastavni backend namjerno još nije bez stanja. Taj nedostatak je vidljiv i označen, a ispravljamo ga u 4. predavanju.

Izvori: The Twelve-Factor App: VI. Processes.

> **Za nastavnika:** Pitajte: smije li proces imati varijablu u memoriji? Očekivano: da, ako njezin gubitak ili razlika među procesima ne mijenja ispravnost (npr. brojač zahtjeva za zapis).

30 Monolit nije pogreška

Vrijeme za nastavu: 85–87 min (2 min).

Naš backend je **monolit**: cijela aplikacija gradi se, pokreće i isporučuje kao jedna cjelina, jedan proces. Riječ se često koristi kao uvreda, ali opisuje način isporuke, a ne kvalitetu koda.

Prednosti su konkretne. Postoji jedno pokretanje i jedan zapis. Poziv između dijelova je poziv funkcije, a ne mrežna poruka koja može kasniti ili se izgubiti. Promjena koja dira narudžbe i proizvode isporučuje se odjednom, a kasnije jedna transakcija može obuhvatiti obje promjene. Cijena: sve se isporučuje zajedno, a bez discipline dijelovi se počnu međusobno pozivati bilo kako i granice nestanu.

Alternativa, raspodjela na više zasebno isporučenih usluga (mikroservisi), pretvara pozive funkcija u mrežne poruke sa svim kvarovima iz 1. predavanja. Za tim i aplikaciju veličine ovog kolegija rijetko se isplati. Kad i zašto bi se isplatila, razmatramo u 12. predavanju.

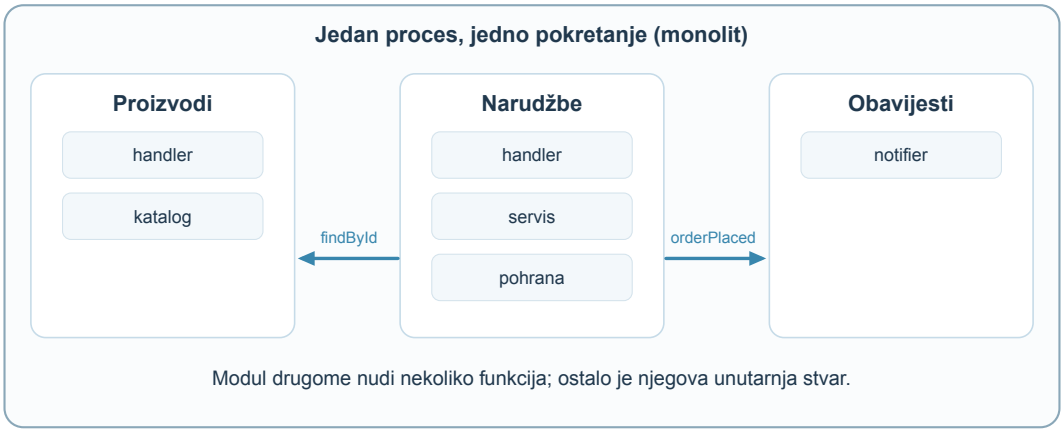
Izvori: Fowler, *MonolithFirst* (2015).

> **Za nastavnika:** Pitajte: koji kvarovi iz 1. predavanja nastaju kad poziv `catalog.findById` postane HTTP zahtjev drugoj usluzi? Očekivano: kašnjenje, izgubljen odgovor, nedostupnost.

31 Modularni monolit

> **Vrijeme za nastavu:** 87–90 min (3 min).

Kako zadržati prednosti monolita bez gubitka granica? **Modularni monolit** dijeli kod po području posla (proizvodi, narudžbe, obavijesti), a ne samo po tehničkim slojevima. Svaki modul ima svoje handlere, pravila i pohranu, a drugima izlaže malo funkcija.



Mape src/products, src/orders i src/notifications u code/02-backend-architecture.

Figure 9: Modularni monolit: jedan proces s tri modula. Narudžbe od proizvoda koriste samo `findById` (i `decreaseStock`), a od obavijesti samo `orderPlaced`.

```
src/products/        katalog, GET /products
src/orders/          handler, servis, pohrana narudžbi
src/notifications/   obavijesti
```

```
src/app.mjs          usmjeravanje
server.mjs           pokretanje, sastavljanje, gašenje
```

Kad netko mijenja proizvode, zna da mora sačuvati funkcije koje koriste narudžbe, a ostalo smije mijenjati slobodno. Ako jednog dana jedan modul treba izdvojiti u zasebnu uslugu, granica već postoji. Podjela po tehničkim slojevima (`controllers/`, `services/`, `repositories/`) također je česta; za male aplikacije obje rade, a podjela po području lakše raste.

> **Za nastavnika:** Otvorite stablo mapa u editoru. Pitajte: kamo bi išao modul za račune (8. predavanje)? Očekivano: `src/invoices/`, s jednom funkcijom koju poziva servis narudžbi ili worker.

32 Isti servis u Fastifyju

> **Vrijeme za nastavu:** 90–93 min (3 min).

Dosad smo usmjeravanje, čitanje JSON-a i gašenje pisali sami da bi se vidjelo što se događa. Framework poput **Fastifyja** te poslove preuzima. Važno je što se **ne** mijenja: servis, katalog, pohrana i obavijest ostaju isti, jer ne ovise o HTTP-u. Mijenja se samo HTTP sloj i pokretanje.

```
// Skica: isti orderService, drugi HTTP sloj (nije dio nastavnog koda)
import Fastify from "fastify";
const app = Fastify({ logger: true });
app.get("/products", async () => catalog.list());
app.post("/orders", async (request, reply) => {
  try {
    return reply.code(201).send(await orderService.placeOrder(request.body));
  } catch (error) {
    if (error instanceof ValidationError) return reply.code(400).send({ error: error.messa
    if (error instanceof OutOfStockError) return reply.code(409).send({ error: error.messa
    throw error;
  }
});
await app.listen({ port: config.port, host: config.host });
process.once("SIGINT", () => app.close());
```

`request.body` je već parsiran JSON; Fastify ga parsira za `Content-Type: application/json`. Neuhvaćenu grešku Fastify pretvara u odgovor `500`. `app.close()` zatvara poslužitelj i čeka započete zahtjeve.

Ovaj isječak nije pokrenut u ovoj verziji materijala (potreban je paket `fastify`); provjeriti ga prije nastave s aktualnom verzijom Fastifyja. Fastify koristimo u kasnijim predavanjima.

Izvori: Fastify: Routes; Fastify: Server; Fastify: Reply.

> **Za nastavnika:** Stavite isječak uz `order-handlers.mjs`. Pitajte: koji redak `order-service.mjs` treba promijeniti za Fastify? Očekivano: nijedan.

33 Što framework preuzima, a što ostaje naše

> **Vrijeme za nastavu:** 93–95 min (2 min).

Posao	Ručno (`node:http`)	Fastify
usmjeravanje	objekt `routes`, 404/405	`app.get`, `app.post`
čitanje JSON tijela	`readJson`	ugrađeno parsiranje
provjera oblika tijela	ručno u servisu	JSON shema na ruti (3. predavanje)
zapisi zahtjeva	`res.on("finish", …)`	`logger: true`
uredno gašenje	`server.close()`	`app.close()`
pravila, podaci, obavijest	**naš kod**	**naš kod**

Framework uklanja ponavljajući HTTP kod i donosi provjerena rješenja za rubne slučajeve. Ne odlučuje gdje su pravila, kako se zaliha ne smije prodati dvaput ni što učiniti kad e-pošta ne radi. Te odluke prenose se u svaki framework, pa i u Spring Boot, ASP.NET Core ili FastAPI.

> **Za nastavnika:** Naglasite prenosivost: student koji razumije ovu tablicu prepoznat će iste dijelove u drugom ekosustavu pod drugim imenima.

34 AI kod na pregledu

> **Vrijeme za nastavu:** 95–99 min (4 min).

AI asistent na upit „napiši servis za narudžbe s e-poštom” predložio je sljedeće. Kod je uvjerljiv i za ispravan zahtjev radi.

```
const EMAIL_API_KEY = "re_8f2a91c4d7…";

export async function placeOrder(req, res) {
  const { email, items, total } = req.body;
  const order = await db.orders.insert({ email, items, total });
  console.log("Narudžba", order, "ključ", EMAIL_API_KEY);
```

```
  await sendEmail(EMAIL_API_KEY, email, order);
  res.status(201).json(order);
}
```

Pronađite barem četiri problema. Za svaki recite koju posljedicu ima i gdje bi taj dio koda trebao biti.

Za nastavnika: Parovi tri minute, zatim prolaz.

Očekivano obrazloženje: (1) Tajna u kodu → konfiguracija iz okruženja. (2) Tajna u zapisu → nikad ne ispisivati; `describeConfig`. (3) `total` dolazi od klijenta → servis računa iz kataloga (Webshop iz 1. predavanja). (4) „Servis" prima `req` i `res` → nije testabilan bez HTTP-a; HTTP pripada handleru. (5) Nema provjere unosa ni zalihe. (6) Ako `sendEmail` baci grešku nakon spremanja, klijent dobiva 500 iako je narudžba spremljena. (7) `db` i `sendEmail` su uvezeni globalno, a ne predani; zamjena u testu nije moguća. Svaki od četiri pronađena je dovoljan.

35 Sažetak

Vrijeme za nastavu: 99–101 min (2 min).

- Jedan handler za `POST /orders` miješa sedam poslova sa sedam razloga za promjenu.
- Usmjeravanje bira handler; HTTP sloj prevodi zahtjev i odgovor; servis drži pravila; repozitorij i obavijest skrivaju podatke i vanjske usluge.
- Pravila ne ovise o detaljima; ovisnosti im predaje `server.mjs`. Zato ih testiramo bez HTTP-a i mijenjamo bazu bez promjene servisa.
- Sloj dodajemo kad izdvaja stvaran razlog za promjenu, ne po navici.
- Konfiguracija dolazi iz okruženja i provjerava se pri pokretanju; tajne nisu u kodu, gitu ni zapisima.
- Proces se pokreće redom i gasi uredno; zajedničko stanje ne smije živjeti u memoriji jednog procesa.
- Monolit je legitiman izbor; modularni monolit čuva granice unutar jednog procesa.

Za nastavnika: Vratite se na prijedloge podjele s početka sata. Pitajte jedan par bi li sada promijenio svoj prijedlog i zašto.

36 Osmislite sljedeći korak

Vrijeme za nastavu: 101–104 min (3 min).

Narudžbe uskoro neće slati samo naša web stranica. Mobilna aplikacija i partnerski sustav također će pozivati `POST /orders`, a te klijente pišu drugi timovi koji ne čitaju naš kod.

Što im moramo obećati? Kako izgleda ispravan zahtjev, koje odgovore i statuse mogu očekivati, kako izgleda greška i što smiju ponoviti ako ne dobiju odgovor?

Dodatni uvjet: usluga e-pošte počinje odgovarati za 30 sekundi. Što se događa s odgovorom na `POST /orders` i s mobilnom aplikacijom koja čeka?

> **Za nastavnika:** Ovo je otvoreni dizajnerski zadatak bez jednog točnog odgovora. Prihvatite prijedloge poput dokumentiranog primjera zahtjeva, popisa statusa ili zajedničkog oblika greške. Na pitanje o ponavljanju dobar je odgovor „ne znamo, ovisi”; to je tema 9. predavanja. Kašnjenje e-pošte vodi prema odvajanju obavijesti od odgovora (8. predavanje). Najavite: sljedeći put pišemo ugovor API-ja (3. predavanje).